

Teil 1: Software Konstruktion

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Softwarekonstruktion (Ingenieurmäßiges Vorgehen):

- Aufgaben, Methoden, Werkzeuge und Anspruch des Ingenieurs

Softwarekonstruktion.

- Die Herausforderungen des Themenfeldes Softwarekonstruktion **aufzählen** und **erläutern**



- Die Aspekte der Softwarekonstruktion **benennen**



- Die Teile des Software Engineering, die nicht zur Softwarekonstruktion gehören, **benennen**



Geschichte der Informatik.

- Die Geschichte des Software Engineering **wiedergeben**



Ingenieurwissenschaft.

- Die Prinzipien ingenieurmäßigen Vorgehens **wiedergeben** und **erläutern**



- Bei der Entwicklung von Software ingenieurmäßiges Vorgehen **planen** und **nutzen**



- Metriken, **kennen**, ihre Vor- und Nachteile **erklären** sowie **anwenden**



Agenda

1. Software Konstruktion

- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

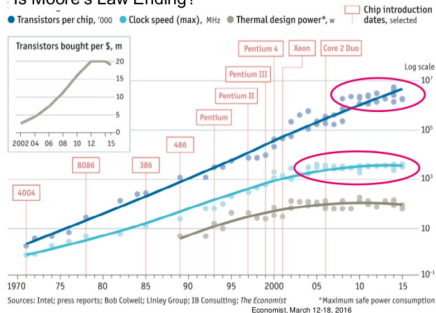
2. Bestandsaufnahme Software Konstruktion

3. Begleitendes Beispiel

Moore's Law und Software

- Erste programmierbare Computer Ende 1940er Jahren
- Erste Programmiersprachen in den 1950er Jahren
- Ab den 1960ern wird Hardware immer günstiger und leistungsfähiger
- Software Projekte werden größer

Is Moore's Law Ending?



Software Krise (I)

Ende der 1960er Jahre

- Software Projekte laufen unbefriedigend ab
- Projekte sprengen Budgets
- Deadlines werden nicht eingehalten
- Software hat Fehler und ist langsam
- Projekte werden abgebrochen

Software Krise (II)

The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.

*Edsger Dijkstra, The Humble Programmer (EWD340),
Communications of the ACM*

Software Krise (III)

Die Idee einer Disziplin „Software Engineering“ entsteht bei einer NATO Konferenz 1968 / 1969 in einer Diskussion der Software Krise

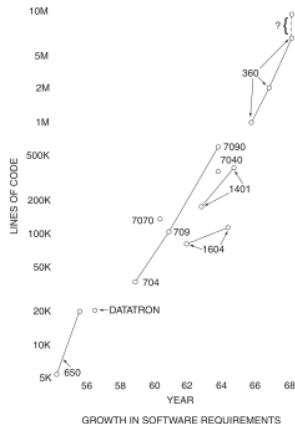
»Production of large software has become a scare item for management. By reputation it is often an unprofitable morass, costly and unending.

...

No less a person than T. J. Watson said that OS/360 cost IBM over 50 million dollars a year during its preparation, and at least 5000 man-years' investment.

...

The commitment to many software projects has been withdrawn. This is indeed a frightening picture.◀



Software Engineering

Software Engineering is...

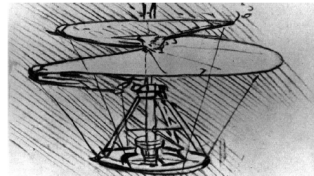


... the establishment and use of **sound engineering principles** in order to obtain **economically** software that is **reliable and works efficiently** on real machines. [NR68]

... the **systematic approach** to the **development, operation, maintenance, and retirement** of software." [IEEE83]

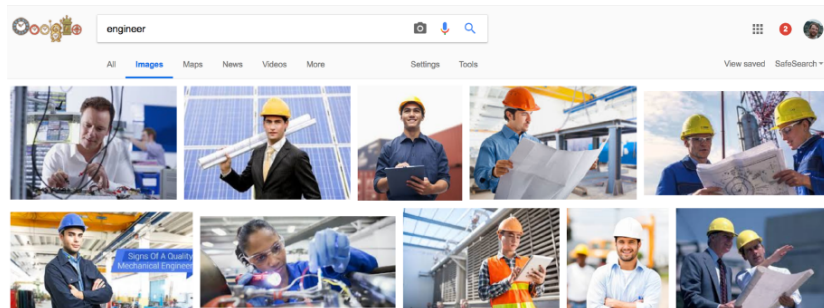


- **Fachleute auf dem Gebiet der Technik**
- Technik: von Menschen gemachte Gegenstände und deren Herstellung und Verwendung
- Das Wort Ingenieur kommt vom lateinischen „ingenium“ Bedeutung ungefähr: **Erfindung, Scharfsinn**
- Der Begriff ist im 16ten Jahrhundert entstanden und hat ursprünglich den **Beruf des künstlerischen Erfinders** beschrieben (z.B. Leonardo da Vinci)



Quelle teilw.: http://www.iaeng.org/about_IAENG.html

Ingenieure



„Engineers *apply the knowledge of the mathematical and natural sciences* (biological and physical), *with judgment and creativity* to *develop ways to utilize the materials and forces of nature* for the benefit of mankind. The subjects are diverse and include names

like bioengineering, computer engineering, electrical and electronics engineering, financial engineering, industrial engineering, internet engineering and systems engineering, etc.“

Ingenieure: Aufgaben und Anspruch

Ingenieure konstruieren komplexe technische Systeme

- Oft interdisziplinär (Physik, Chemie, Biologie)
- Auch für neue Produkte: traditionell vorhandene Methoden und Mittel zur Herstellung
- Ingenieur beherrscht Herstellungsmethoden, Werkzeuge und Werkstoffe (Regeln, Standards, Erfahrungswissen)

Dreifacher Anspruch

- **Produkt:** „utilize the materials and forces of nature“
 - Technischer Anspruch
- **Methoden:** „with judgment and creativity“
 - Empirischer Anspruch
- **Ingenieur:** „for the benefit of mankind“
 - Ethischer Anspruch

Beispiel: Hausbau

<https://www.youtube.com/watch?v=tlnUVxqAzDU>

Praxis: Abwägung Kosten / Nutzen

Das Teufelsquadrat im Projektmanagement

- Optimierung einer Dimension geht zu Lasten der anderen Dimensionen
- Generell unmöglich, alle Dimensionen zu optimieren

Strategien für die Abwägung von Kosten und Nutzen

- Schätzen
- Messen

← Maße
Metriken



Teufelsquadrat nach Harry Sneed

Messen / Auswahl von Metriken

Wer viel misst, misst Mist

Goal Question Metric Paradigm (GQM)

Goal:

Was ist das Ziel?

Question:

Was muss dazu analysiert werden?

Metric:

Was ist eine aussagekräftige Metrik?

Beispiel:

Kontrolle der Kosten

Analyse:

Erwartete Gesamtkosten

Metrik / Maß:

Bisherige Kosten / Fertigstellungsgrad

Praxis: Beherrschung von Komplexität

Strategien für die Beherrschung von Komplexität

Abstraktion:

- Einsatz von geeigneten Modellen
- Ausblenden von Detailinformation

Spezialisierung / Modularisierung:

- Aufteilung in klar abgegrenzte Unterstrukturen
- Partitionierung der Aufgaben (Dekomposition)

Standardisierung:

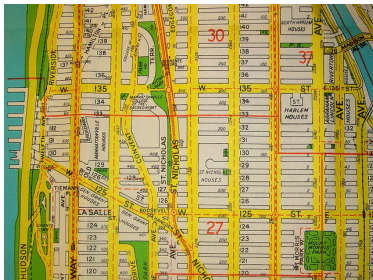
- Dokumentation von Erfahrungswissen
- Einfache Integration und Wiederverwendung

Werkzeuge:

- Erhöhung von Leistungsfähigkeit und Präzision

Abstraktion

Abstrakt (Plan / Modell):



Konkret:



<https://www.flickr.com/photos/23465812@N00/7968397600/>

<https://www.flickr.com/photos/eirikref/2302281782/>

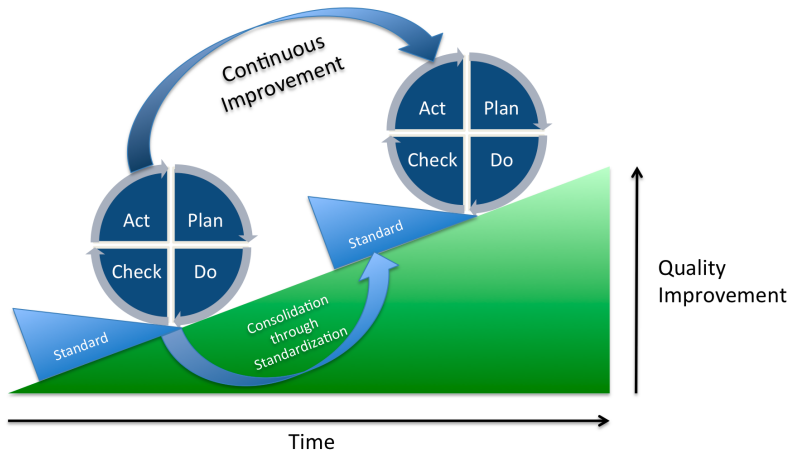
<https://creativecommons.org/licenses/by/2.0/>

Spezialisierung / Modularisierung



<https://www.flickr.com/photos/martindew/7945608932/>
<https://creativecommons.org/licenses/by/2.0/>

Standardisierung



Beispiele: Stecker, Schrauben, Gewinde, Vorgehen, Maße, ...

Werkzeuge



<https://flic.kr/p/fkmB7T>, (CC BY 2.0)

Zusammenfassung Ingenieure

- Konstruieren komplexe technische Systeme
- Nutzen Wissen aus den Naturwissenschaften
- Dreifacher Anspruch
(Produkt, Methoden, Ingenieur bzw. Technisch, Empirisch, Ethisch)
- Wägen Kosten gegen Nutzen ab (Schätzen und Messen)
- Benötigen Strategien zum Umgang mit Komplexität (Abstraktion, Spezialisierung / Modularisierung, Standardisierung, Werkzeuge)

Software Engineering (II)

Abstraktion,
Spezialisierung,
Modularisierung,
Standardisierung,
...



Software Engineering is...

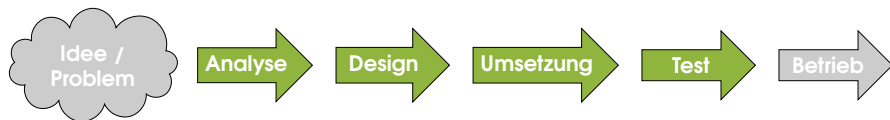
... the establishment and use of **sound engineering principles** in order to obtain economically software that is reliable and works efficiently on real machines. [NR68]

Schätzen, Messen, ...



... the systematic approach to the development, operation, maintenance, and retirement of software." [IEEE83]

Software Konstruktion



Methoden, Sprachen, Vorgehen

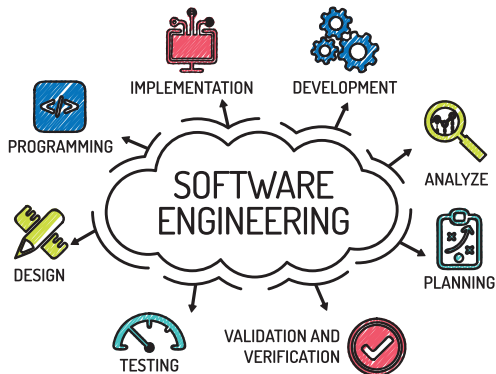
Sprachen, Modelle

Werkzeuge

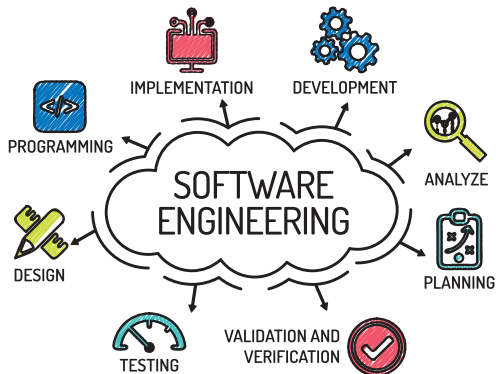
Standards

Qualität, Kennzahlen (Produkt / Prozess)

SWK Teilgebiete (Spezialisierung)



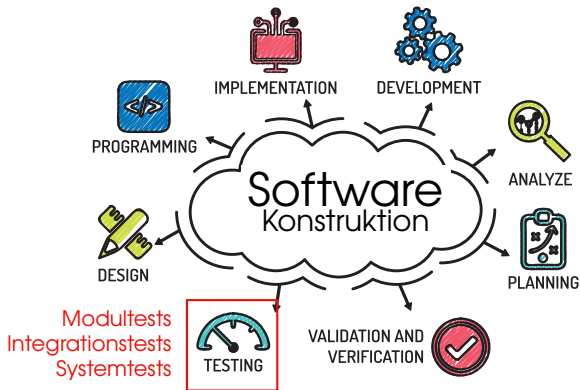
SWK Teilgebiete (Spezialisierung)



Software Engineering:

Marketing, Organisation, Multi-Projekt Management, SWK, Wartung, Support, Dokumentation, ...

SWK Teilgebiete (Spezialisierung)



Software Engineering:

Marketing, Organisation, Multi-Projekt Management, SWK, Wartung, Support, Dokumentation, ...

SWK Geschichte (Werkzeuge)

Auszug:

- 1968 Dijkstra "goto" ist gefährlich
- frühe 1970er Strukturierte Programmierung, erste Objektorientierung
- späte 1970er Erste IDEs
- späte 1980er CASE Werkzeuge
- späte 1980er Weitere Verbreitung von OOP durch C++ und OOD
- frühe 1990er Kommerzielle Werkzeuge für Anforderungsmanagement
- später 1990er Java, UML, ...
- frühe 2000er Eclipse IDE, CASE Werkzeuge verschwinden

Beispiel: Sprachen (Komplexität)

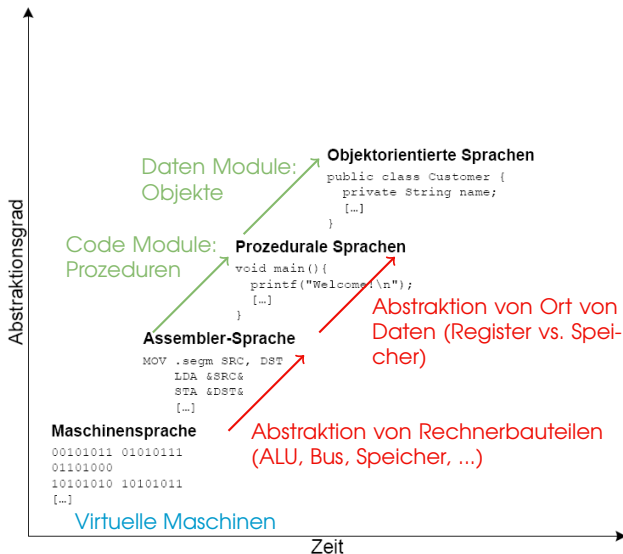
- Problem:
Komplexitäts-
Beherrschung

- Lösung:

Abstraktion

Modularisierung

Standards



Beispiel: SCRUM Report (Messen)

Support from senior management can make or break Scrum practice. This metric is reported as the most likely point of tension for Scrum teams, and the most important consideration for organizations adopting Scrum. The good news, however, is that support from senior management is the least likely to be a challenge for Scrum implementation.

Important considerations when adopting scrum

Active senior management and support

57%

Alignment with strategic and financial goals of the company

52%

Participation of experienced trainers and coaches

44%

Clear set of business goals to be achieved

43%

Challenges with implementing Scrum

Organizational design and culture made it difficult to adopt and scale

51%

Difficult to transition from traditional Waterfall

44%

No clearly defined metrics to identify and measure success

41%

Alignment with other projects in portfolio

40%

Lack of trust

38%

Desire for perceived predictability

35%

Product owners/teams not willing/enthusiastic

34%

Fear of transparency

28%

Had to convince clients

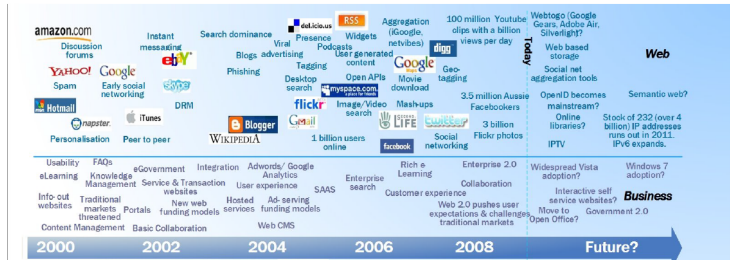
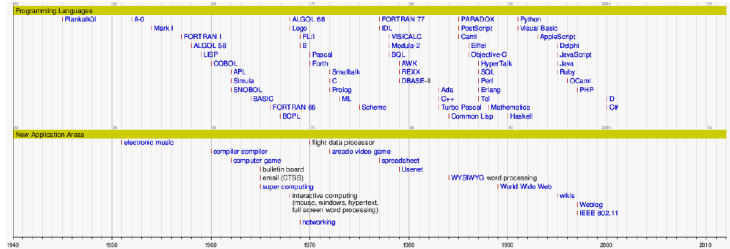
24%

No support from senior management

19%

[https://www.scrumalliance.org/ScrumRedesignDEVSite/media/ScrumAllianceMedia/Files%20and%20PDFs/State%20of%20Scrum/2017-SoSR-Final-Version-\(Pages\).pdf](https://www.scrumalliance.org/ScrumRedesignDEVSite/media/ScrumAllianceMedia/Files%20and%20PDFs/State%20of%20Scrum/2017-SoSR-Final-Version-(Pages).pdf)

Geschichte (Domänen, Paradigmen, Werkzeuge)



Zusammenfassung Geschichte

- Neue Technologien (z.B. Web)
- Neue Klassen von Anwendungen (z.B. Apps)

→ Neue Paradigmen, Werkzeuge und Methoden (z.B. SCRUM)

→ Domänen-spezifische Lösungen (Kampfjet vs. Web-Anwendung)

ABER:

Generell gültige Ansprüche an Produkt, Methoden und Ingenieur

Anspruch an Software

- Korrektheit
- Zuverlässigkeit
- Sicherheit
- Effizienz
- Benutzbarkeit
- Wartbarkeit
- Lesbarkeit
- Erweiterbarkeit
- Wiederverwendbarkeit

Anspruch an Methoden

- Stellt Anforderungen an Software sicher
- Effektiv
- Effizient
- Planbar



Anspruch an Software Ingenieur

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

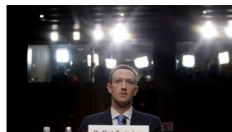
The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

1. PUBLIC – Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER – Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT – Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT – Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT – Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION – Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES – Software engineers shall be fair to and supportive of their colleagues.
8. SELF – Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Facebook / Cambridge Analytica

- Cambridge Analytica „klaut“ Nutzerdaten von Facebook
- Erstellt psychologische und politische Profile der Nutzer (Nix: 4.000 bis 5.000 Datenpunkte zu jedem Erwachsenen US Amerikaner)
- Profile werden Basis für Werbekampagnen vor US Wahl 2016
- Trump wird überraschend Präsident



Fakt oder Fiktion?

Zusammenfassung Software Konstruktion

- Software Konstruktion zielt ab auf die **ingenieurmäßige** Entwicklung, Wartung, Anpassung und Weiterentwicklung **großer Softwaresysteme**.
- Verwendung **bewährter systematischer Vorgehensweisen**, Prinzipien, Methoden und Werkzeuge.
- Empirisches Vorgehen (**Ausprobieren, Beobachten, Standardisieren**)
- Methoden und Vorgehen teilweise spezifisch für Domäne.
- Berücksichtigung der Aspekte: **Funktionalität, Kosten, Zeit, Qualität**.

Agenda

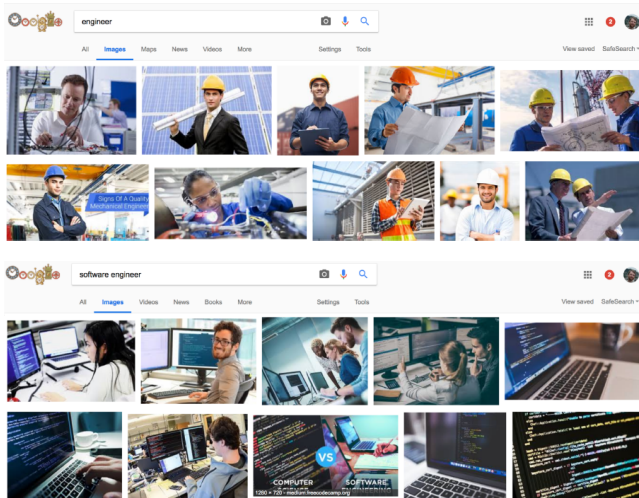
1. Software Konstruktion

- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

2. Bestandsaufnahme Software Konstruktion

3. Begleitendes Beispiel

Engineer vs. Software Engineer



Bestandsaufnahme (I)

- Entwicklungsprozesse der Software-Industrie bei weiten nicht so planbar, zuverlässig, effektiv, effizient und flexibel wie die „althergebrachter“ Industrien:
- Keine zuverlässige Herstellung von Software im industriellen Maßstab.
- Kosten- und Terminüberschreitungen.
- Bei Auslieferung: ungenügende Softwarereife.
- Keine Produktivitätskontrolle wie in anderen industriellen Fertigungsbereichen.
- Keine Qualitätskontrolle wie in anderen industriellen Fertigungsbereichen, gerade das Testen ist immer noch unterbewertet.

Bestandsaufnahme (II)

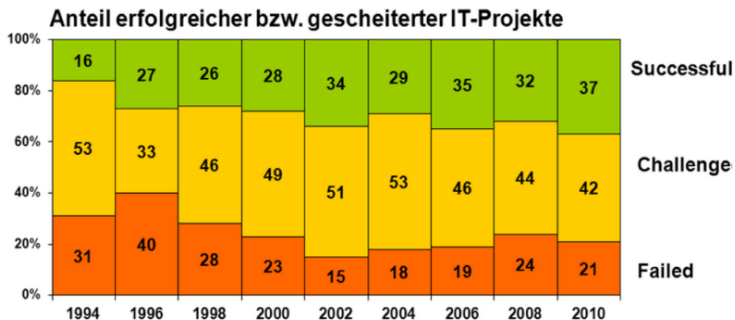
Anzahl Function	Früher als geplant	Termingerecht	Verspätet	Abgebrochen
1 FP	14,86%	83,16%	1,92%	0,25%
10 FP	11,08%	81,25%	5,67%	2,00%
100 FP	6,06%	74,77%	11,83%	7,33%
1.000 FP	1,24%	60,76%	17,67%	20,33%
10.000 FP	0,14%	28,03%	23,83%	48,00%
100.000 FP	0,00%	13,67%	21,33%	65,00%
Durchschnitt	5,53%	56,94%	13,71%	23,82%

Abbruchrate großer Projekte nach [Jon96]

Andere Quelle: **Erfolgreich** durchgeführte **Software-Projekte**:
16% im Jahre 1994; 34% im Jahre 2003

1 Function Point (FP): ca. 55 Lines of Code (LOC) in C++/Java, 20 LOC Perl, 13 LOC SQL, etc.
Desktop-Projekt mit 1 FP dauert ca. eine Woche mit einem Entwickler
Allgemeines Projekt mit 100.000 FP und 100 Entwicklern: ca. 17 Monate

Bestandsaufnahme (III)



Standish Group – **Chaos Report**

Erfolgreich durchgeführte Software-Projekte:
35% seit 15 Jahren
Gescheiterte Software-Projekte:
20%

Abnahme gescheiterter Projekte:
Geänderte Kriterien

Andererseits ...

Bauvorhaben

- Flughafen BER
 - Eröffnungstermine: 2007, 2011, 2012, 2013, 2019 ...
 - Tatsächlich: 31.10.2020
- Elbphilharmonie
- Stuttgart 21
 - Baubeginn: 2010
 - Aktuelle Planung: Ende 2025

Software

- Falcon 9 Rakete landet auf Fuß:



Image: <https://i.imgur.com/j463wtT.gifv>

Video: <https://www.youtube.com/watch?v=V4-w4wV3aVQ>

Gründe

- Software ist viel komplexer als alles, was wir als Menschen bisher konstruiert haben
- Software verändert sich immer noch stark
- Software Engineering ist eine junge Disziplin
- (Immer noch) Großer Teil der Entwickler von Software hat keine Ausbildung als Software Ingenieur

Meine Erfahrungen mit Software Projekten

Agenda

1. Software Konstruktion

- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

2. Bestandsaufnahme Software Konstruktion

3. Begleitendes Beispiel

SWK Anwendungsbeispiel

Remote Home Control

- Geschäftsidee: Kunden können Geräte im Haus kontrollieren und steuern.
- Größeres Software Projekt: Sensoren, Aktuatoren, Server, App, ...

Dieses Jahr ohne echte Software :(



<https://flic.kr/p/oStisy> (CC BY 2.0)